



Project Name: Round Robin vs Priority comparison Project

Course: Operating System

Team Members:

رغد إيهاب محمود

بسملة إبراهيم السيد

جني ماهر ناصر

زيزف اشرف جمال

اوليفيا ناجي فوزي

حبيبة كمال رمضان

روان سعد عيد

Date: 10/5/2026

Table Of Contents

Introduction:	3
Rules:	4
Scenario A: Basic mixed workload:	5
Scenario B: Urgency case.....	6
Scenario C: Fairness & Rotation	7
Scenario D: The Starvation Risk	8
Scenario E: Validation Case.....	9
Project Assumptions & Limitations:	10
Conclusion:	10

Introduction:

This project provides a **computational synthesis** which bridges the gap between theoretical operating system principles and practical application by converting raw process data into dynamic visualizations and comparative metrics that evaluate the trade-offs between **Urgency and Fairness**.

1. Dual-Algorithm Comparison

The core of the system is its ability to run and compare two distinct algorithms:

- **Round Robin:** It treats every process equally by giving each a tiny "time slice" (Quantum) and rotating through them.
- **Priority Scheduling (Preemptive):** It ignores the "wait your turn" rule and instead executes tasks based on their priority or urgency level.

2. Dynamic Input Management

The system doesn't just run a static script; it acts as a flexible data handler. It allows users to:

- **Input at Runtime:** Define a custom number of processes, their arrival times, and what Burst Time (Running Time) they need.
- **Set Constraints:** Manually adjust the **Time Quantum** for the Round Robin algorithm and assign **Priority Values**.



The screenshot shows the 'CPU Scheduling Simulator' interface. It features a dark blue background with white and light blue text. At the top, the title 'CPU Scheduling Simulator' is displayed with a gear icon. Below the title, there are two main sections. The first section, 'Add Process', includes a '+ Add Process' button and a form with four input fields: 'Process ID', 'Arrival Time', 'Burst Time', and 'Priority'. An 'Add' button is positioned to the right of these fields. The second section, 'Settings & Scenarios', includes a 'Round Robin Quantum' input field with the value '3'. Below this, there are five buttons labeled 'A: Standard', 'B: Urgency', 'C: Fairness', 'D: Starvation', and 'E: Validation'.

3. Real-Time Logic Validation:

In Scenario E, it specifically demonstrates how the system handles **invalid data** without crashing, showing the robustness of the code.

4. Visual Gantt Charts

Instead of just giving you a wall of text, the system translates mathematical execution orders into **Gantt Charts**.

- It visually maps out which process has control of the CPU.
- This makes it easy to see exactly when a high-priority process "preempts" (kicks out) a lower-priority one.

5. Mathematical Performance Analytics

The system serves as a calculator for "Efficiency Metrics." For every process, it automatically computes:

- **Waiting Time (WT):** How long a task sat in the queue.
- **Turnaround Time (TAT):** Total time to execute a specific process.
- **Response Time (RT):** How quickly the CPU first acknowledged the task.
- **Averages:** It then averages these numbers to provide a high-level "score" for how well the algorithm performed overall.

6. Behavioral Analysis (The "Stress Test")

The system is designed to highlight specific OS phenomena:

- **Starvation Check:** It shows how low-priority tasks can be "starved" of CPU time if high-priority tasks keep arriving.
- **Fairness Assessment:** It demonstrates how Round Robin prevents starvation by forcing rotation, even if it makes high-priority tasks wait longer.

Rules:

Turn Around Time (TAT)

= Completion Time(CT) – Arrival Time (AT)

= Waiting Time(WT) + Burst Time (BT)

Waiting Time (WT)= Turnaround Time - Burst Time

Response Time (RT) =First Start Time - Arrival Time

Scenario A: Basic mixed workload:

How do algorithms handle a basic 1-2-3 queue?

Preemptive Analysis: Since all arrive at 0, they execute purely by rank: P2 (High) -> P3 (Med) -> P1 (Low).

Round Robin Analysis (q=3): Every process gets a fair 3s slice. Notice P1 and P2 appear twice because their bursts exceed the quantum.

Settings & Scenarios

Round Robin Quantum:

A: StandardB: UrgencyC: FairnessD: StarvationE: Validation

Processes Queue

ID	Arrival	Burst	Priority	
P1	0	5	3	Edit Delete
P2	0	4	1	Edit Delete
P3	0	3	2	Edit Delete

Run Simulation

Clear All

Scenario A: Normal Workload (Normal)

Goal: How do algorithms handle a basic 1-2-3 queue?

Analysis: Every process gets a fair 3s slice. Notice P1 and P2 appear twice because their bursts exceed the quantum.

Analysis: Since all arrive at 0, they execute purely by rank: P2 (High) -> P3 (Med) -> P1 (Low).

Round Robin

Avg WT: **6.67** | Avg TAT: **10.67**

Avg RT: **3.00**

Gantt Chart

Preemptive Priority

Avg WT: **3.67** | Avg TAT: **7.67**

Avg RT: **3.67**

Gantt Chart

Winner Algorithm

Preemptive Priority is more efficient for this case

Scenario B: Urgency case

The Question: P2 (High Prio) arrives at Time 2 while P1 is running. Where does P3 fit?

Preemptive Analysis: Preemptive: P1 is kicked out at Time 2 so P2 can finish. P3 waits until both are done.

Round Robin Analysis (q=3): P2 and P3 enter the queue as they arrive, taking their turns after P1's first slice.

Settings & Scenarios

Round Robin Quantum:

A: StandardB: UrgencyC: FairnessD: StarvationE: Validation

Processes Queue

ID	Arrival	Burst	Priority	
P1	0	8	2	Edit Delete
P2	2	2	1	Edit Delete
P3	3	2	3	Edit Delete

Run Simulation

Clear All

Scenario B: The Urgency Test (Preemption)

Goal: P2 (High Prio) arrives at Time 2 while P1 is running. Where does P3 fit?

Analysis: P2 and P3 enter the queue as they arrive, taking their turns after P1's first slice.

Analysis: Preemptive: P1 is kicked out at Time 2 so P2 can finish. P3 waits until both are done.

Round Robin

Avg WT: **2.33** | Avg TAT: **6.33**

Avg RT: **1.00**

Gantt Chart

0 1 2 3 4 5 6 7 8 9 10 11 12

Preemptive Priority

Avg WT: **3.00** | Avg TAT: **7.00**

Avg RT: **2.33**

Gantt Chart

0 1 2 3 4 5 6 7 8 9 10 11 12

Winner Algorithm

Round Robin is more efficient for this case

Scenario C: Fairness & Rotation

The Question: Does the CPU rotate effectively with three processes?

Preemptive Analysis: P1 hogs the CPU for 6s, then P2, then P3. Low-priority P3 waits a full 12s to start.

Round Robin Analysis ($q=3$): ensures fairness by granting every process an equal time quantum of 3 units, cycling through each in order so no process is starved or prioritized over another.

Settings & Scenarios

Round Robin Quantum:

A: StandardB: UrgencyC: FairnessD: StarvationE: Validation

Processes Queue

ID	Arrival	Burst	Priority	
P1	0	6	1	Edit Delete
P2	0	6	2	Edit Delete
P3	0	6	3	Edit Delete

Run Simulation

Clear All

Scenario C: Fairness & Rotation

Goal: Does the CPU rotate effectively with three processes?

Analysis: ensures fairness by granting every process an equal time quantum of 3 units, cycling through each in order so no process is starved or prioritized over another.

Analysis: P1 hogs the CPU for 6s, then P2, then P3. Low-priority P3 waits a full 12s to start.

Round Robin

Avg WT: **9.00** | Avg TAT: **15.00**

Avg RT: **3.00**

Gantt Chart

Process	Start	End
P1	0	3
P2	3	6
P3	6	9
P1	9	12
P2	12	15
P3	15	18

Preemptive Priority

Avg WT: **6.00** | Avg TAT: **12.00**

Avg RT: **6.00**

Gantt Chart

Process	Start	End
P1	0	6
P2	6	12
P3	12	18

Winner Algorithm

🏆 Preemptive Priority is more efficient for this case

Scenario D: The Starvation Risk

The Question: Can multiple high-prio tasks block a low-prio task?

Preemptive Analysis: Starvation: P3 is 'trapped' behind P1 and P2 because they both have higher priority.

Round Robin Analysis ($q=3$): P3 finishes at Time 8, which is much faster than waiting until Time 15 in the priority charts.

Settings & Scenarios

Round Robin Quantum:

A: StandardB: UrgencyC: FairnessD: StarvationE: Validation

Processes Queue

ID	Arrival	Burst	Priority	
P1	0	10	1	Edit Delete
P2	1	5	1	Edit Delete
P3	2	2	5	Edit Delete

Run SimulationClear All

Scenario D: The Starvation Risk

Goal: Can multiple high-prio tasks block a low-prio task?

Analysis: P3 finishes at Time 8, which is much faster than waiting until Time 15 in the priority charts.

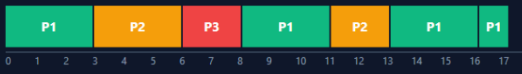
Analysis: Starvation: P3 is 'trapped' behind P1 and P2 because they both have higher priority.

Round Robin

Avg WT: **6.00** | Avg TAT: **11.67**

Avg RT: **2.00**

Gantt Chart

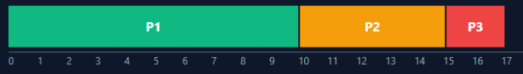


Preemptive Priority

Avg WT: **7.33** | Avg TAT: **13.00**

Avg RT: **7.33**

Gantt Chart



Winner Algorithm

🏆 **Round Robin is more efficient for this case**

Scenario E: Validation Case

The Question: What happens when invalid process data is submitted to the scheduler?

For both: ⚠️ **Invalid Input: No processes were submitted. Priority Scheduling requires at least one process with a valid arrival time, burst time, and priority value to begin execution.**

Settings & Scenarios

Round Robin Quantum:

A: StandardB: UrgencyC: FairnessD: StarvationE: Validation

Processes Queue

ID	Arrival	Burst	Priority
----	---------	-------	----------

Run SimulationClear All

Scenario E: Validation Check

Goal: How the system handles empty inputs?

⚠️ Error: Round Robin queue is empty.

⚠️ Error: No processes provided for Priority Scheduling.

Round Robin

Gantt Chart

Preemptive Priority

Gantt Chart

Winner Algorithm

Project Assumptions & Limitations:

- **Zero Context-Switching:** To focus on algorithmic logic, the time taken by the CPU to switch between processes is assumed to be zero.
- **Preemptive Nature:** The Priority algorithm is fully preemptive, meaning it re-evaluates the CPU owner at every 1-unit time step.
- **No Aging:** The current Priority implementation does not include "Aging," which intentionally demonstrates the potential for **Starvation** in low-priority tasks.

Conclusion:

After executing and analyzing the five test scenarios, the following conclusions were reached:

- **Round Robin (RR):** Proved to be the most effective algorithm for ensuring **Fairness**. It prevents starvation and provides a balanced response time for all processes regardless of their size or priority.
- **Preemptive Priority:** Demonstrated superior performance in handling **Urgent** tasks. It ensures that critical processes are served immediately, though at the risk of delaying lower-priority tasks (Starvation).
- **Summary:** The simulator successfully visualized these trade-offs, proving that the choice of scheduling algorithm depends entirely on the system's goals (e.g., real-time responsiveness vs. general-purpose fairness).